

Oracle Database 19c Backup & Recovery · 제16장

RMAN 고급 복구와 블록 손상 복구

RECOVER TABLE · spfile 복구 · 블록 손상 감지와 판정 · Block Media Recovery · Data Recovery Advisor

학습 목표 (Objectives)

- 이 장을 마치면 다음을 수행할 수 있어야 한다.
 - 테이블 하나만 과거 시점으로 되살린다
 - spfile과 pfile 손상을 복구한다
 - 블록 손상을 감지하고 원인을 판정한다
 - 손상된 블록만 복구한다
 - Data Recovery Advisor로 장애를 진단하고 조치한다
 - 블록 손상을 조기에 탐지하는 설정을 적용한다

이 장의 구성 (Agenda)

- 16.1 테이블 단위 복구 — 선택지 · 동작 · 제약 · 로그 읽기
- 16.2 파라미터 파일 복구 — spfile/pfile · 자동 백업 · 값 오류 · 점검
- 16.3 블록 손상의 감지와 판정 — 종류 · 감지 · 조기 탐지 · 백업 중 손상
- 16.4 블록 미디어 복구 — RECOVER BLOCK · CORRUPTION LIST · 우회 · 비교
- 16.5 Data Recovery Advisor — LIST/ADVISE/REPAIR · 주의
- 16.6 판단표 · 16.7 퀴즈 · 요약 · 실습 8 개요

참고:

15장까지의 실습이 완료되어 백업과 아카이브가 준비된 상태를 전제로 한다.

16.1

테이블 단위 복구

- 테이블 하나를 과거 시점으로 되살린다
- 보조 인스턴스와 Data Pump를 내부에서 사용
- REMAP TABLE을 기본으로 쓴다

복구 방법의 선택

시점과 UNDO가 기준

상황	권장 방법
----- -----	
몇 분 전 DELETE, UNDO 있음	Flashback Query / Flashback Table
DROP 직후, 휴지통에 있음	Flashback Drop
며칠 전 시점, UNDO 없음	RECOVER TABLE
TRUNCATE 후 시간 경과	RECOVER TABLE
테이블스페이스 전체가 대상	TSPITR

-- Flashback은 UNDO에 의존하고 RECOVER TABLE은 백업과 아카이브를 쓴다.

RECOVER TABLE

명령 하나, 다섯 단계

```
RMAN> RECOVER TABLE hr.emp160
  2>  UNTIL TIME "TO_DATE('2025-05-18 09:00:00','YYYY-MM-DD HH24:MI:SS')"
```

```
  3>  AUXILIARY DESTINATION '/u03/aux'
```

```
  4>  REMAP TABLE hr.emp160:emp160_recovered;
```

- ① 보조 인스턴스를 임시로 생성
- ② 그 시점의 SYSTEM·SYSAUX·UNDO와 대상 TS 복원
- ③ 보조 인스턴스를 그 시점까지 복구해 오픈
- ④ Data Pump로 대상 테이블 추출
- ⑤ 원본으로 가져온 뒤 보조 인스턴스 정리

제약과 준비 사항

- 대상이 될 수 없는 것
 - SYS 스키마의 테이블
 - SYSTEM.SYSAUX 테이블스페이스의 테이블
- 전제 조건
 - ARCHIVELOG 모드
 - 지정 시점의 SYSTEM.SYSAUX.UNDO 백업이 모두 있을 것
- 공간 — 대상 테이블이 작아도 세 TS만큼 필요

수행 로그 읽기

다섯 구간

Creating automatic instance, with SID='qBxA'

① 생성

contents of Memory Script:

② 자동 구성 스크립트

```
{  
  set until time ...;  
  restore clone controlfile;  
  recover clone database tablespace 'SYSTEM','UNDOTBS1','SYSAUX','USERS';  
}
```

EXPDP> . . exported "HR"."EMP160" 8.789 KB 107 rows

③ 추출

IMPDP> . . imported "HR"."EMP160_RECOVERED" 107 rows

④ 반입

Removing automatic instance

⑤ 정리

16.2

파라미터 파일 복구

- spfile은 자동 백업에 함께 들어간다
- 복원 순서가 까다롭다
- 값 오류는 pfile을 경유해 고친다

spfile 복원

임시 pfile → SET DBID → RESTORE

```
RMAN> STARTUP NOMOUNT
startup failed: ORA-01078: failure in processing system parameters
LRM-00109: could not open parameter file '.../dbs/initiorcl.ora'
```

-- 최소 pfile을 만든다

```
[oracle@oel7v9r1 ~]$ echo "db_name=orcl" > /tmp/init_tmp.ora
```

```
RMAN> STARTUP NOMOUNT PFILE='/tmp/init_tmp.ora';
RMAN> SET DBID 1200530011;
RMAN> RESTORE SPFILE FROM AUTOBACKUP;
channel ORA_DISK_1: SPFILE restore from AUTOBACKUP complete
```

```
RMAN> SHUTDOWN IMMEDIATE
RMAN> STARTUP
```

값 오류로 기동 불가

pfile을 경유해 수정

```
SYS@orcl> ALTER SYSTEM SET sga_target = 40G SCOPE = SPFILE;
System altered.

SYS@orcl> SHUTDOWN IMMEDIATE
SYS@orcl> STARTUP
ORA-00821: Specified value of sga_target is too small
-- 또는 메모리 부족으로 기동 실패

SYS@orcl> CREATE PFILE='/tmp/fix.ora' FROM SPFILE;
SYS@orcl> !sed -i 's/^\*.sga_target=.*/*sga_target=1073741824/' /tmp/fix.ora
SYS@orcl> STARTUP PFILE='/tmp/fix.ora';
SYS@orcl> CREATE SPFILE FROM PFILE='/tmp/fix.ora';
SYS@orcl> SHUTDOWN IMMEDIATE
SYS@orcl> STARTUP
```

복구 후 파라미터 점검

- 복원본에는 자동 백업 이후의 변경이 없다
- 반드시 확인할 항목
 - log_archive_dest — 아카이브 경로
 - control_files — 컨트롤파일 경로
 - db_recovery_file_dest — FRA 경로와 크기
 - 메모리 관련 값 — 서버 사양과 맞는지
- v\$parameter 의 isdefault='FALSE' 목록으로 확인

16.3

블록 손상의 감지와 판정

- 물리적 손상과 논리적 손상
- 감지 경로는 네 가지
- 조기 탐지가 복구 가능성을 좌우한다

손상의 감지

네 가지 경로

-- ① 사용자 조회 중 (가장 늦은 발견)

ORA-01578: ORACLE data block corrupted (file # 7, block # 142)

ORA-01110: data file 7: '/u02/oradata/ORCL/users01.dbf'

ORA-26040: Data block was loaded using the NOLOGGING option -- 원인 표시

-- ② RMAN 백업 중 자동 감지 ③ VALIDATE 명령

RMAN> VALIDATE DATAFILE 7;

File	Status	Marked Corrupt	Empty Blocks	Blocks Examined	High SCN
------	--------	----------------	--------------	-----------------	----------

7	FAILED	3	384	1280	2690400
---	--------	---	-----	------	---------

-- ④ DBVERIFY (데이터베이스 밖에서)

[oracle@oel7v9r1 ~]\$ dbv file=/u02/oradata/ORCL/users01.dbf blocksize=8192

Total Pages Marked Corrupt : 3

조기 탐지 설정

CPU와 맞바꾼다

```
SYS@orcl> SHOW PARAMETER db_block_check
```

NAME	TYPE	VALUE

db_block_checking	string	FALSE
db_block_checksum	string	TYPICAL

-- 권장 설정

```
SYS@orcl> ALTER SYSTEM SET db_block_checksum = FULL SCOPE = BOTH;
```

```
SYS@orcl> ALTER SYSTEM SET db_block_checking = MEDIUM SCOPE = BOTH;
```

-- db_block_checksum : OFF / TYPICAL(기본) / FULL

-- db_block_checking : OFF / LOW / MEDIUM / FULL

백업 중 손상을 만났을 때

MAXCORRUPT

```

RMAN> BACKUP DATAFILE 7;
RMAN-03009: failure of backup command
ORA-19566: exceeded limit of 0 corrupt blocks for file
/u02/oradata/ORCL/users01.dbf
-- 기본 허용치가 0이므로 손상 하나에도 실패한다.

RMAN> RUN {
  2> SET MAXCORRUPT FOR DATAFILE 7 TO 10;
  3> BACKUP DATAFILE 7;
  4> }
-- 10개까지는 기록하고 계속 진행한다.

-- BACKUP CHECK LOGICAL 로 논리 손상까지 검사하며 백업할 수도 있다.
```

손상 발견 후의 판정

범위·원인·방법

판정 조건	조치
----- -----	
블록 몇 개, 백업 있음	RECOVER ... BLOCK
블록 다수, 파일 전반	파일 단위 RESTORE/RECOVER
CORRUPTION_TYPE이 NOLOGGING	재적재 (복구 불가)
같은 파일에서 반복 발생	디스크 교체 후 파일 복구
백업에도 손상 블록 포함	더 오래된 백업 + 아카이브 적용

```
SYS@orcl> SELECT owner, segment_name, segment_type FROM dba_extents
2  WHERE file_id = 7 AND 142 BETWEEN block_id AND block_id + blocks - 1;
-- 손상 블록이 인덱스라면 REBUILD가 더 빠르다.
```

16.4

블록 미디어 복구

- 손상된 블록만 되돌린다
- 업무를 멈추지 않는다
- NOLOGGING 유형은 복구되지 않는다

RECOVER BLOCK

두 가지 방식

-- 특정 블록 지정

```
RMAN> RECOVER DATAFILE 7 BLOCK 142;
```

```
RMAN> RECOVER DATAFILE 7 BLOCK 142 TO 144;
```

```
RMAN> RECOVER DATAFILE 7 BLOCK 142, 150, 201;
```

-- 손상 목록 전체를 한 번에

```
RMAN> VALIDATE DATABASE;
```

```
RMAN> RECOVER CORRUPTION LIST;
```

```
Starting recover at 18-MAY-25
```

```
channel ORA_DISK_1: restoring block(s)
```

```
starting media recovery
```

```
media recovery complete, elapsed time: 00:00:02
```

```
SYS@orcl> SELECT COUNT(*) FROM v$database_block_corruption; -- 0
```

블록 복구와 파일 복구 비교

판단 기준

구분	블록 복구	파일 복구
대상	손상된 블록만	파일 전체
읽는 양	KB 단위	파일 크기 전체
소요 시간	수 초	크기에 비례
OFFLINE 필요	불필요	필요
업무 영향	그 블록을 읽는 세션만	해당 TS 전체 중단
적합한 상황	손상 수십 개 이하	손상이 파일 전반

복구가 불가능한 경우와 우회

- 블록 복구가 성립하지 않는 조건
 - 백업에도 같은 블록이 손상된 경우
 - 아카이브가 결손된 경우
 - NOLOGGING으로 적재되어 리두에 내용이 없는 경우
- 임시 우회 — DBMS_REPAIR.skip_corrupt_blocks
 - 손상 블록을 건너뛰고 조회한다 (그 행은 손실)
 - 마지막 수단이며 복구 가능해지면 즉시 되돌린다

16.5

Data Recovery Advisor

- 감지된 장애를 목록으로 관리한다
- 조치안과 스크립트를 제시한다
- 판단은 여전히 사람의 몫

LIST / ADVISE / REPAIR

네 명령의 흐름

```

RMAN> LIST FAILURE;
Failure ID Priority Status      Time Detected Summary
-----
142          HIGH      OPEN        18-MAY-25    One or more non-system
                                     datafiles are missing

RMAN> ADVISE FAILURE;
Optional Manual Actions
1. If file ... was unintentionally renamed or moved, restore it
Automated Repair Options
1      Restore and recover datafile 7
      Strategy: complete media recovery with no data loss

RMAN> REPAIR FAILURE PREVIEW;      -- 스크립트 확인
RMAN> REPAIR FAILURE;              -- 실행

```

DRA 제시안과 15장 절차

특별한 방법이 아니다

감지된 장애	DRA 제시 스크립트
----- -----	
일반 데이터파일 부재	DATAFILE OFFLINE → RESTORE → RECOVER → DATAFILE ONLINE
SYSTEM 데이터파일 부재	(MOUNT 요구) RESTORE → RECOVER
컨트롤파일 부재	RESTORE CONTROLFILE FROM AUTOBACKUP → MOUNT → RECOVER → OPEN RESETLOGS
블록 손상	RECOVER ... BLOCK
온라인 리두 그룹 부재	ALTER DATABASE CLEAR LOGFILE GROUP n

DRA를 안전하게 쓰는 방법

- LIST FAILURE로 상황 파악까지만 쓴다
- ADVISE FAILURE로 조치안을 참고한다
- REPAIR FAILURE PREVIEW로 내용을 확인한다
- 직접 판단한 뒤 필요하면 스크립트를 수정해 실행
- 여러 장애가 있으면 CRITICAL부터
- 해결된 장애는 CHANGE FAILURE ... CLOSE 로 정리

고급 복구 판단표

범위를 좁힐수록 영향이 적다

상황	조치	영향
테이블 하나, 며칠 전 시점	RECOVER TABLE ... UNTIL	없음
테이블 하나, 몇 분 전	Flashback (18장)	없음
TS 전체, 과거 시점	TSPITR	해당 TS
spfile 손상	임시 pfile + RESTORE SPFILE	전면 중단
파라미터 오류로 기동 불가	pfile 변환 후 수정	전면 중단
블록 몇 개 손상	RECOVER ... BLOCK	없음
손상 목록 전체	RECOVER CORRUPTION LIST	없음
NOLOGGING 블록	재적재	해당 객체
블록 다수 손상	파일 단위 복구	해당 파일
진단이 막막할 때	LIST/ADVISE FAILURE	없음

수동 절차와의 대응

17장에서 손으로 수행한다

RECOVER TABLE이 없던 시절의 절차

- ① 별도 서버에 복제 데이터베이스 생성
- ② 그 시점까지 복구해 오픈
- ③ Data Pump로 테이블 추출
- ④ 원본으로 반입
- ⑤ 복제본 삭제

지금도 수동 절차가 필요한 경우

SYS 스키마나 SYSTEM TS의 객체를 되살릴 때
보조 인스턴스를 같은 서버에 만들 수 없을 때
여러 테이블을 각각 다른 시점으로 되살려야 할 때

파라미터 파일 관리 원칙

세 겹의 대비

- ① 컨트롤파일 자동 백업 - RMAN으로 복원 가능
- ② pfile 텍스트 사본 - 변경 때마다 남긴다
- ③ 형상 관리 저장소 - 변경 이력 추적

```
[oracle@oel7v9r1 ~]$ sqlplus -s / as sysdba <<EOF
> CREATE PFILE='/home/oracle/backup/initorc1_$(date +%Y%m%d).ora' FROM SPFILE;
> EOF
```

```
[oracle@oel7v9r1 ~]$ diff /home/oracle/backup/initorc1_20250510.ora \
> /home/oracle/backup/initorc1_20250518.ora
```

-- 값이 언제 어떻게 바뀌었는지 추적할 수 있다.

복구 범위와 소요 시간

좁힐수록 영향이 적다

복구 범위	예상 소요	업무 영향
블록 몇 개	수 초	없음 (해당 블록 조회만)
테이블 하나	수십 분	없음 (별도 이름으로)
데이터파일 하나	크기에 비례	해당 TS 중단
테이블스페이스	크기에 비례	해당 TS 중단
데이터베이스 전체	전체 크기 비례	전면 중단

판단에 쓰는 조회

<code>v\$database_block_corruption</code>	- 손상 블록의 수와 유형
<code>v\$recover_file</code>	- 복구가 필요한 파일
<code>dba_extents</code>	- 손상 블록이 속한 객체

퀴즈 1

RECOVER TABLE의 내부 동작

RECOVER TABLE이 내부적으로 사용하는 것은?

Flashback 영역

보조 인스턴스와 Data Pump

Data Guard

카탈로그

퀴즈 2

NOLOGGING 블록의 처리

CORRUPTION_TYPE이 NOLOGGING인 블록의 처리 방법은?

RECOVER BLOCK으로 복구

리두에 내용이 없으므로 재적재

DBMS_REPAIR로 복구

파일 복구

퀴즈 3

REPAIR FAILURE 전 PREVIEW

REPAIR FAILURE를 바로 실행하지 않고 PREVIEW를 먼저 보는 이유는?

속도가 빨라서
제시안이 손실을 수반할 수 있어 내용 확인이 필요하므로
권한 때문에
로그 기록 때문

요약

- RECOVER TABLE은 보조 인스턴스와 Data Pump를 내부에서 쓴다
- REMAP TABLE을 기본으로 써 현재 테이블을 덮어쓰지 않는다
- spfile은 임시 pfile로 기동한 뒤 자동 백업에서 복원한다
- 파라미터 값 오류는 pfile로 변환해 수정한다
- 블록 손상은 VALIDATE와 DBVERIFY로 감지한다
- 블록 몇 개면 RECOVER BLOCK이 압도적으로 빠르다
- DRA는 진단을 돕지만 판단을 대신하지 않는다

이 장의 결론

- 복구 범위를 좁히는 것이 이 장의 주제
 - 테이블 하나 · 블록 몇 개 · 파라미터 파일
- 좁힐수록 업무 영향이 줄지만 판정이 정확해야 한다
- 판정에 쓰는 조회 세 가지를 절차서에 적어 둔다
- 도구가 자동화해도 판단은 사람의 몫
 - DRA의 제시안은 참고 자료다
- 다음 장 — 복제로 같은 일을 손으로 수행

실습 16 개요

- 16-1 RECOVER TABLE · 16-2 보조 인스턴스 관찰
- 16-3 spfile 복구 · 16-4 파라미터 값 오류 복구
- 16-5 블록 손상 재현과 VALIDATE
- 16-6 Data Recovery Advisor
- 16-7 RECOVER BLOCK · 16-8 DBVERIFY와 조기 탐지

실습 준비:

16-1과 16-2는 보조 인스턴스용으로 수 GB의 여유 공간이 필요하다.